

Máster en Inteligencia Artificial
para Liderazgo Técnico y Transformación Empresarial

Costes de Infraestructura y TCO para Sistemas LLM

Módulo 9 · Infraestructura y Despliegue

Modelo de coste completo, routing por complejidad, cuantización, prefix caching y break-even cloud
vs GPU.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Costes de Infraestructura y TCO para Sistemas LLM

El coste de infraestructura es frecuentemente el mayor inhibidor del escalado de sistemas LLM en empresa. Un sistema que cuesta \$500/mes con 10.000 consultas mensuales puede costar \$50.000/mes con 1.000.000 de consultas, si no se optimiza correctamente. La diferencia entre un sistema económicamente sostenible a escala y uno que colapsa financieramente no está en el modelo elegido: está en las decisiones de arquitectura de coste.

Este tema cubre el análisis completo del TCO (Total Cost of Ownership) de sistemas LLM en producción: los componentes del coste (infraestructura de inferencia, embeddings, reranking, monitorización, almacenamiento), las estrategias de optimización más efectivas, y cómo construir un modelo de coste que permita proyectar el gasto con diferentes escenarios de crecimiento.

Los componentes del coste de un sistema LLM

Coste de inferencia: la partida dominante

El coste de inferencia del LLM (tokens de entrada + tokens de salida) representa típicamente el 60-80% del coste operativo total de un sistema LLM basado en API de proveedor. Los precios orientativos de abril 2025: GPT-4o (\$2.50 entrada/\$10 salida por M tokens), Claude 3.5 Sonnet (\$3/\$15), GPT-4o-mini (\$0.15/\$0.60), Claude 3 Haiku (\$0.25/\$1.25), Gemini 1.5 Flash (\$0.075/\$0.30). Para un sistema con 1M de tokens de entrada y 200K de tokens de salida diarios, el coste mensual en GPT-4o es $\sim \$75 + 60 = \$135/\text{día} \times 30 = \$4.050/\text{mes}$.

Coste de embeddings y retrieval

En sistemas RAG, el embedding de las consultas (para búsqueda vectorial) es una partida de coste adicional. text-embedding-3-large: \$0.13/M tokens. Voyage-3-large: \$0.06/M tokens. Para 1M de consultas de 50 tokens de media: 50M tokens de embedding = \$6.50 con Voyage vs \$6.50 con OpenAI. El coste de la base vectorial (Pinecone, Weaviate cloud) se añade: Pinecone Starter (gratuito hasta 1M vectores), Pinecone Standard desde \$70/mes. El coste del reranking (Cohere Rerank: \$2/M tokens) también contribuye si se aplica a cada consulta.

Coste de infraestructura de GPU autohosteada

Para sistemas con modelos autohosteados (vLLM + modelo open source en GPU cloud), el coste es por hora de GPU: A100 80GB en CoreWeave: $\sim \$2.21/\text{hora}$, H100 SXM en CoreWeave: $\sim \$2.49/\text{hora}$. Un sistema con 2x A100 para alta disponibilidad, 24/7: $2 \times \$2.21 \times 24 \times 30 = \$3.182/\text{mes}$ más el coste del sistema operativo, almacenamiento, y networking ($\sim \$200\text{-}500/\text{mes}$). Para modelos Llama 3 70B, 2x A100 con vLLM pueden manejar $\sim 100\text{-}150$ req concurrentes.

Estrategias de optimización de coste

Routing por modelo según complejidad

La estrategia de mayor impacto en optimización de coste es el routing por complejidad: clasificar cada consulta según su dificultad y enviarla al modelo más económico que puede manejarla. Un clasificador ligero (puede ser el propio GPT-4o-mini) categoriza las consultas en "simple", "media", "compleja". Las simples (70% del tráfico típicamente) van a GPT-4o-mini o Gemini Flash. Las complejas (10-15%) van al modelo frontier más capaz. Una empresa que implementó este routing reportó una reducción del 42% del coste total manteniendo la misma calidad percibida por los usuarios.

Instancias reservadas vs on-demand

Para workloads predecibles en cloud, las instancias reservadas reducen el coste un 30-50% vs on-demand. AWS Reserved Instances con compromiso de 1 año: hasta 40% de descuento. Spot Instances para workloads tolerantes a interrupciones (batch processing, fine-tuning): hasta 90% de descuento vs on-demand. Para los pods del warm pool con tráfico predecible, las instancias reservadas son casi siempre más económicas que on-demand si el factor de utilización es >50%.

Cuantización para reducir requisitos de GPU

La cuantización reduce la VRAM necesaria 2-8x con pérdida de calidad mínima (<1-5%). Llama 3 70B en FP16: 140GB VRAM (requiere 2x A100 80GB). En INT8: 70GB VRAM (cabe en 1x A100 80GB). En Q4 GGUF: 35GB VRAM (cabe en 1x A100 40GB o RTX 4090). Reducir de 2x A100 a 1x A100 equivale a un ahorro de $\$2.21/\text{hora} \times 24 \times 30 = \$1.591/\text{mes}$. Para modelos que toleran Q4 cuantización (la mayoría para tareas de QA y clasificación), el ahorro es sustancial.

SECCIÓN 4 · EJEMPLOS REALES

- Modelo de coste detallado para asistente empresarial (10.000 usuarios, 50 consultas/usuario/mes = 500.000 consultas/mes, prompt promedio 1.500 tokens + 500 tokens de respuesta): sin optimización con GPT-4o: 750M tokens entrada $\times \$2.50$ + 250M tokens salida $\times \$10$ = $\$1.875$ + $\$2.500$ = $\$4.375/\text{mes}$. Con routing (70% a GPT-4o-mini, 30% a GPT-4o): $\$1.500$ + $\$1.000$ = $\$2.500/\text{mes}$ (-43%). Con semantic caching al 40% de hit rate: consultas efectivas al modelo = 300.000 $\rightarrow$ $\$1.500/\text{mes}$ (-66% respecto al inicio).
- Optimización de costes de RAG en plataforma de análisis legal: el corpus de documentos recuperados representaba el 60% de los tokens de entrada (3.000 de 5.000 tokens promedio). Reduciendo de K=10 a K=3 documentos con reranking previo: tokens de entrada reducidos de 5.000 a 2.600 promedio. Ahorro del 48% en tokens de entrada. Mejora adicional: algunos documentos recuperados eran idénticos entre consultas del mismo expediente $\rightarrow$ prefix caching del 35% de los tokens de contexto.
- Break-even cloud API vs GPU cloud para startup: 2.000.000 consultas/mes con Llama 3 8B equivalente en calidad a GPT-4o-mini. Con GPT-4o-mini API: $\$1.200/\text{mes}$. Con 1x A100 en CoreWeave: $\$1.591/\text{mes}$. Con scale-to-zero en horario no laboral (50% de las horas): $\$795/\text{mes}$. Break-even a 2M consultas/mes con scale-to-zero.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: No incluir los costes de monitorización, evaluación, y almacenamiento en el TCO. Las llamadas al LLM-as-judge para evaluación continua (5% del volumen), el almacenamiento de logs de auditoría, el coste de la plataforma LLMOps (LangSmith, Langfuse), y el coste de embeddings para retrieval pueden sumar el 15-30% del coste de inferencia principal. El TCO real incluye todos estos componentes.

Error 2: Optimizar el coste de tokens sin evaluar el impacto en la calidad. Reducir el contexto RAG de 10 a 3 documentos puede reducir el coste un 40% pero también puede reducir la calidad en casos donde los documentos 4-10 son necesarios para responder correctamente. Siempre evalúa el impacto en las métricas de calidad (RAGAS faithfulness, context recall) antes de aplicar optimizaciones de coste agresivas.

Error 3: No revisar los costes mensualmente. Los precios de los modelos de los proveedores cambian frecuentemente (la competencia ha reducido los precios un 70-90% en muchos modelos en 2024-2025). Un modelo que costaba \$0.15/M tokens cuando se implantó el sistema puede costar \$0.05/M seis meses después. La revisión mensual de la estructura de costes y comparación con alternativas puede producir ahorros significativos.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

La plantilla de modelo de coste mensual para sistemas LLM: (1) Coste de inferencia = volumen_mensual × tokens_por_consulta × precio_por_token. (2) Coste de embeddings = volumen_mensual × tokens_por_query × precio_embedding. (3) Coste de GPU autohosteada = horas/mes × precio/hora × num_GPUs. (4) Coste de base vectorial = plan o precio por vector. (5) Coste de monitorización = 5% del volumen × precio_evaluacion. (6) Coste de almacenamiento = GB_logs/mes × precio_almacenamiento. Suma todos los componentes para el TCO mensual.

Para implementar el routing por complejidad: define tres tiers de consulta (simple, media, compleja). Construye un clasificador: puede ser un prompt a GPT-4o-mini preguntando "¿Esta consulta requiere razonamiento complejo? Responde solo: simple, media, o compleja." El coste del clasificador es mínimo (<\$0.001 por consulta en GPT-4o-mini) y el ahorro en las consultas simples ruteadas a modelos baratos justifica ampliamente la inversión.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M8-T5 (Gestión de tokens y costes): el coste de tokens de inferencia es el componente más grande del TCO.
- M9-T1 (Cloud vs on-premise): el análisis de TCO es la base de la decisión de despliegue.
- M9-T5 (Escalabilidad): el semantic caching y el routing reducen el coste además de mejorar la escalabilidad.

SECCIÓN 8 · RESUMEN EJECUTIVO

El TCO de sistemas LLM incluye: inferencia (60-80%), embeddings y retrieval, GPU o API, monitorización y evaluación (10-15%), y almacenamiento. Las estrategias de optimización de mayor impacto son: routing por complejidad (-40-60% de coste manteniendo calidad), semantic caching (-30-60% de consultas al modelo), cuantización

(hasta 4x reducción de GPU necesaria), prefix caching del system prompt (-30-40% de tokens de entrada), instancias reservadas (-30-50% de GPU cloud para workloads predecibles). Revisa mensualmente: los precios del mercado caen frecuentemente. El TCO real siempre incluye monitorización, evaluación continua y almacenamiento.